DSU W24 MODEL ANSWER MSBTE

This Study Material is provided by Campusifyplus.in ©

Made by Team Campusify

THIS IS THE FREE STUDY MATERIAL BY CAMPUSIFY!!

Be Part of Our Community

Telegram Channel - https://t.me/Campusifyy

Whatsapp Channel - https://tinyurl.com/Campusify-channel

Campusify is building the Biggest Diploma/Engineering Student Community in Maharashtra. Be part of the movement!

Join Channel for MSBTE Study Materials



Earn by Referring Campusify Course

Presented by **Campusify** | **10K** Active Members | **2.7L** Students Helped within **1 year**

Refer Our course with your **Third year** friend or cousin and get **10%** commission on each referral.

For More Details : WhatsApp Us at **+91 8291934640**

TY Course Link - https://www.campusifyplus.site

MAHARASHTRA STATE BOARD OF TECHNICAL EDUCATION
(Autonomous)
(ISO/IEC - 27001 - 2013 Certified)

Model Answer

WINTER– 24 EXAMINATION

Subject Name: Data Structure using 'C' Subject Code: **313301**

Important Instructions to examiners:

1) The answers should be examined by key words and not as word-to-word as given in the model answer scheme.
2) The model answer and the answer written by candidate may vary but the examiner may try to assess the understanding level of the candidate.
3) The language errors such as grammatical, spelling errors should not be given more importance (Not applicable for subject English and Communication Skills).
4) While assessing figures, examiner may give credit for principal components indicated in the figure. The figures drawn by candidate and model answer may vary. The examiner may give credit for any equivalent figure drawn.
5) Credits may be given step wise for numerical problems. In some cases, the assumed constant values may vary and there may be some difference in the candidate's answers and model answer.
6) In case of some questions credit may be given by judgement on part of examiner of relevant answer based on candidate's understanding.
7) For programming language papers, credit may be given to any other program based on equivalent concept.

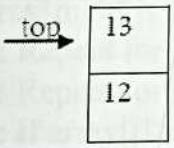
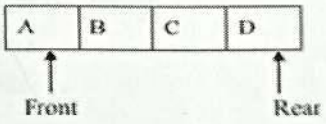
Q. No	Sub Q.N.	Answer	Marking Scheme
1.		**Attempt any five:**	10 Marks
	a) Ans:	**Write any four operations performed on data structure.** **Operations on data structure:** Insertion Deletion Searching Sorting Traversing Merging	
	b) Ans:	**List any two applications of stack.** 1. Backtracking 2. Reversing a string 3. Polish notations 4. Processing Function Calls 5. Parentheses Checking	

6. String Reversal(Recursion)
7. Syntax Parsing
8. Memory Management
9. Conversion of infix to prefix
10. Evaluation of postfix expression
11. Conversion of infix to prefix expression
12. Evaluation of prefix expression

c) Ans: **List any four sorting techniques.**

1. Bubble Sort
2. Selection Sort
3. Insertion Sort
4. Quick Sort
5. Merge Sort

d) Ans: **Differentiate between stack and queue.(Min.two points each)**

Stack	Queue
1. Stack is a data structure in which insertion and deletion operations are performed at **same end**.	1. Queue is a data structure in which insertion and deletion operations are performed at **different ends**.
2. In stack an element inserted last is deleted first so it is called **Last In First Out list.**	2. In Queue an element inserted first is deleted first so it is called **First In First Out list.**
3.In stack only one pointer is used called as **stack top** 4. **Example**: Stack of books	3.In Queue two pointers are used called as **front** and **rear** 4. **Example**: Students standing in a line at fees counter
5.**Application:** • Recursion • Polish notation	5. **Application:** • In computer system for organizing processes. • In mobile device for sending receiving messages.
6. **Representation**: Using array 	6. **Representation:** Using array 

e) Ans: **State the following terms:**

i. **Leaf Node :** The node of the tree, which doesn't have any child node, is called a leaf node. A leaf node is the bottom-most node of the tree. There can be any number of leaf nodes present in a general tree. Leaf nodes can also be called external nodes or Terminal nodes.

ii. **Degree of a Tree:** The degree of tree is the maximum degree of nodes in the tree.

	f) Ans:	**State the syntax of declaration of doubly linked list.** struct Node { int data; struct Node* prev; struct Node* next; }*start=NULL; **struct Node**: A structure that contains the node's data, the previous node, and the next node Node prev*: A pointer to the previous node Node next*: A pointer to the next node	
	g) Ans:	**Define the term general tree and binary tree.** • **General tree :** A general tree is a tree where each node may have zero or more children. Or A general tree is defined as a non empty set T of elements called nodes,such that 1. A set of zero items is a tree,called the empty tree. 2. If T1,T2,....,Tn are n trees for n>0.Within T,R is called the root of T and T1,T2,...,Tn are called sub trees. • **Binary tree**: It is a nonlinear data structure in which each non-leaf node can have maximum two child nodes as left child ad right child.	
2		**Attempt any three:**	**12 Marks**
	a) Ans:	**Describe Bubble sort. State its advantage and disadvantages.** **1) BUBBLE SORT :** Bubble sort is a simple sorting algorithm. This sorting algorithm is comparison-based algorithm in which each pair of adjacent elements is compared and the elements are swapped if they are not in order. This algorithm is not suitable for large data sets as its average and worst case complexity is of O (n2) where **n** is the number of items. **2) Bubble Sort Algorithm:** Let Array[max] is array,N is size of array. **Step 1:** Repeat for pass=1 to N-1. **Step 2:** Repeat for j=0 to N-pass-1. **Step 3:** If array[j]>array[j+1] **Step 4:** InterchangeArray[i] withArray[j+1]. **Step 5:** Print sorted array **3) Bubble Sort Working:** We take an unsorted array for our example as A[]={19, 2, 27, 3, 7, 5, 31}. Bubble sort takes O(n2) time so we're keeping it short and precise. **Pass 1**: 2,19,27,3,7,5,31 2,19,27,3,7,5,31 2,19,3,27,7,5,31	

2,19,3,7,27,5,31
2,19,3,7,5,27,31
Pass 1 Completed

Pass 2: 2,19,3,7,5,27,31
2,3,19,7,5,27,31
2,3,7,19,5,27,31
2,3,7,5,19,27,31
2,3,7,5,19,27,31
Pass 2 Completed

Pass 3: 2,3,7,5,19,27,31
2,3,7,5,19,27,31
2,3,5,7,19,27,31
Pass 3 Completed

Pass 4: 2,3,5,7,19,27,31
Pass 4 Completed

Pass 5: 2,3,5,7,19,27,31
Pass 5 Completed

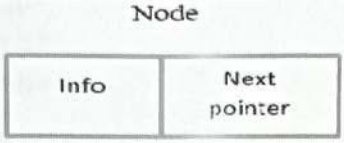
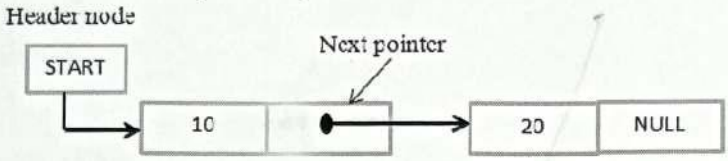
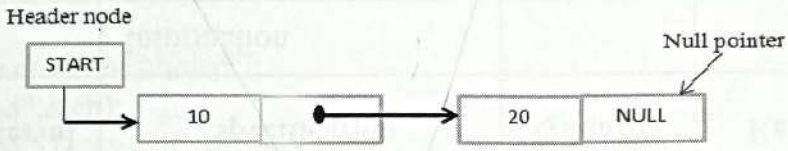
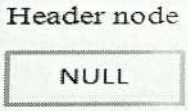
Pass 6: 2,3,5,7,19,27,31
Pass 6 Completed

4) Advantages:

1) It is simple algorithm easy to understand and implement.
2) It is beginner friendly algorithm and can be good for explanatory purposes.
3) The space complexity for bubble sort algorithm is constant, O(1). No additional data or memory is required, it is an inplace sorting algorithm.
4) It can be adapted easily.
5) It is a stable algorithm, which means that it preserves the relative order of equal elements found in the array.
6) It does not require any additional memory space.

- **Disadvantages:**

1) The time complexity of the bubble sort algorithm is quadratic, O(n2), where N is the size of array.
2) It is not suitable for large datasets. As the element in array increases the performance of bubble sort decreases.
3) It is very less used in the practical world.

	b)	**Describe the concept of linked list with the terminologies: node, next pointer null pointer sand empty list.**	
	Ans:	A linked list is a linear data structure that stores a sequence of elements. Each element in the list is called a node, and each node has a reference to the next node in the list. The first node in the list is called the head, and the last node in the list is called the tail. 1. **Node**: Each data element in a linked list is represented as a node. Node contains two parts one is info (data) and other is next pointer (address). Info part stores data and next pointer stores address of next node.  2. **Next pointer**: It is a pointer that holds address of next node in the list i.e. next pointer points to next node in the list.  3. **Null pointer**: It is a pointer that does not hold any memory address i.e. it is pointing to nothing. It is used to specify end of the list. The last element of list contains NULL pointer to specify end of list.  4. **Empty list**: Each linked list has a header node. When header node contains NULL value, then that list is said to be empty list. 	
	c)	**Explain types of queue and implement a program for circular queue.**	
	Ans:	**Types of queue:** 1. **Simple Queue or Linear Queue:** Simple Queue is a linear data structure that follows the First-In-First-Out principle. It allows elements to be added/inserted to the end of the queue (rear) and remove from the front. 2. **Circular Queue:** In Circular queue, all nodes are represented as a circular. It is similar to the linear queue accept that the last element of the queue is connected to the first element. It is also known as a ring buffer as all the ends are connected to	

another end.

3. **Priority Queue:** Elements are arranged by priority, with the element with the lowest priority leaving the queue first.
4. **Double Ended Queue (or Deque):** Double ended queue is a linear data structure in the queue insertion and deletion can be done from the both ends of the queue either from the front or rear ends of queue.

Write a 'C' Program to perform INSERT and DELETE operations on Circular Queue using an Array.

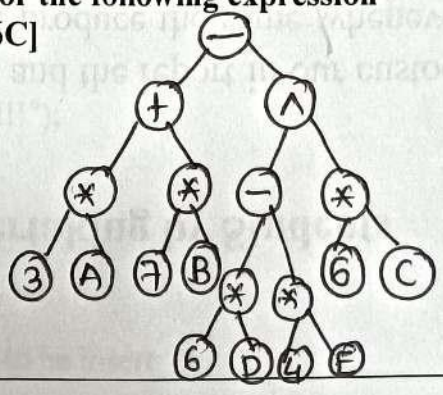
```
#include<stdio.h>
#include<conio.h>
#include<stdlib.h>
#define MAX 5
int queue[MAX],front=-1,rear=-1;
void enqueue();
void dequeue();
void display();
int main()
{
int ch;
clrscr();
printf("\n**circular queue menu**");
printf("\n 1.insert\n 2.delete\n 3.display\n 4.exit");
while(1)
{
printf("\n enter the choice");
scanf("%d",&ch);
switch(ch)
{
case 1:enqueue();
break;
case 2:dequeue();
break;
case 3: display();
break;
case 4:exit(0);
default:
printf("wrong choice");
}}}
void enqueue()
{
if((front==rear+1)||(front==0 && rear==MAX-1))
printf("\n queue is full");
else
{
if(front==-1)
```

```
front=rear=temp;
else
{
rear->next=temp;
rear=temp;
}
}
void dequeue()
{
if(front==NULL)
printf("\n queue is empty!!\n");
else
{
struct node*temp=front;
front=front->next;
printf("\n deleted element :%d\n",temp->data);
free(temp);
}
}
void display()
{
if(front==NULL)
printf("\n queue is empty!!");
else
{
struct node*temp=front;
while(temp->next!=NULL)
{
printf("%d",temp->data);
temp=temp->next;
}
printf("%d",temp->data);
}
}
```

```
front=0;
rear=(rear+1)%MAX;
printf("\n enter element to insert");
scanf("%d",&queue[rear]);
printf("\n after insertion position of front=%d and
rear=%d",front,rear);
}}
void dequeue()
{
if(front==-1)
printf("\n queue is empty");
else
{
printf("deleted element is %d",queue[front]);
if(front==rear)
{
front=-1;
rear=-1;
}
else
front=(front+1)%MAX;
}
printf("After deleting position of front=% and rear=%d",front,rear);
}
void display()
{
int i;
if(rear==-1)
printf("\n queue is empty");
else
{
printf("**element is queue**");
for(i=front;i!=rear;i=(i+1)%MAX)
printf("\n%d",queue[i]);
printf("\n %d",queue[i]);
}}
```

d) **Draw tree structure for the following expression**

Ans: **(3A +7B)-[(6D-4E)^ 6C]**



3		Attempt any three:	12 Marks
	a)	**Write "C" program to implement a linear queue with insert and delete operation.**	
	Ans:	Write a 'C' Program to perform Insert and Delete Operations on Linear Queue using an Array.	

```
#include<stdio.h>
#include<conio.h>
#include<stdlib.h>
#define MAX 5
int queue[MAX],rear=-1,front=-1;
void enqueue();
void dequeue();
void display();
void main()
{
int ch;
clrscr();
printf("**** Queue Menu *****");
printf("\n1.Insert\n2.Delete\n3.Display\n4.Exit");
while(1)
{
printf("\nEnter the choice:");
scanf("%d",&ch);
switch(ch)
{
case 1:enqueue();
break;
case 2:dequeue();
break;
case 3:display();
break;
case 4:exit(0);
default:
printf("\nWrong choice");
}}}
void enqueue()
{
if(rear==MAX-1)
printf("\n Queue is full!!!");
else
{
if(front==-1)
front=0;
rear++;
printf("\nEnter element to be insert:");
```

```
scanf("%d",&queue[rear]);
}}
void dequeue()
{if(front==-1)
printf("Queue is empty!!!\n");
else
{
printf("Deleted element is:%d\n",queue[front]);
front++;
if(front>rear)
front=rear=-1;
}}
void display()
{
int i;
if(rear==-1)
printf("Queue is empty!!!!");
else
{
printf("***** Element in Queue *****\n");
for(i=front;i<=rear;i++)
printf("%d\n",queue[i]);
}}
```

OR

Write a 'C' Program to perform Insert and Delete operations on Linear Queue using a Linked List.

```
#include<stdio.h>
#include<conio.h>
#include<stdlib.h>
#include<process.h>
struct node
{
int data;
struct node*next;
}*front=NULL,*rear=NULL;
void enqueue();
void dequeue();
void display();
int main()
{
int choice;
printf("\n**queue using linked list**\n");
printf("1.insert\n 2. delete\n 3.display\n 4.exit\n");
while(1)
```

```
{
printf("\n enter your choice:(1.insert 2.delete 3.display 4.exit)");
scanf("%d",&choice);
switch(choice)
{
case 1:enqueue();
break;
case 2:dequeue();
break;
case 3:display();
break;
case 4:exit(0);
default:
printf("wrong choice");
}}}
void enqueue()
{
int value;
printf("enter the element to be inserted:");
scanf("%d",&value);
struct node*temp;
temp=(struct node*)malloc(sizeof(struct node));
temp->data=value;
temp->next=NULL;
if(front==NULL)
front=rear=temp;
else
{
rear->next=temp;
rear=temp;
}}
void dequeue()
{
if(front==NULL)
printf("\n queue is empty!!\n");
else
{
struct node*temp=front;
front=front->next;
printf("\n deleted element :%d\n",temp->data);
free(temp);
}
}
void display()
{
if(front==NULL)
```

```
printf("\n queue is empty!!");
else
{
struct node*temp=front;
while(temp->next!=NULL)
{
printf("%d",temp->data);
temp=temp->next;
}
printf("%d",temp->data);
}
}
```

b) **Convert following expression into post fix form. Give stepwise procedure.**
A*(B+C)/D-G

Ans:

Given Expression: A*(B+C)/D-G

Stack	Input	Output
Empty	A * (B + C) / D – G	–
Empty	* (B + C) / D – G	A
*	(B + C) / D – G	A
*(	B + C) / D – G	A
*(	+ C) / D – G	AB
*(+	C) / D – G	AB
*(+	) / D – G	ABC
*	/ D – G	ABC+
/	D – G	ABC+*
/	– G	ABC+*D
–	G	ABC+*D/
–	End	ABC+*D/G
Empty	End	ABC+*D/G–

Postfix Expression : ABC+*D/G-

c) Differentiate between linear data structure and non-linear data structure.(Min.four points each.)

Ans:

Linear Data Structure	Non-Linear Data Structure
1. In a linear data structure, data elements are arranged in a linear order.	1. In a non-linear data structure, data elements are attached in hierarchically manner.
2. Single level is involved.	2. Whereas in non-linear data structure, multiple levels are involved.
3. In a linear data structure, memory is not utilized in an efficient way.	3. While in a non-linear data structure, memory is utilized in an efficient way.
4. Its examples are: array, stack, queue, linked list, etc.	4. While its examples are: trees and graphs.
5. Linear data structures are useful for simple data storage and manipulation.	5. Non-linear data structures are useful for representing complex relationships and data hierarchies, such as in social networks, file systems, or computer networks.
6. The linear data structures are comparatively easier to implement.	6. The non-linear data structures are comparatively difficult to implement and understand.
7. Linear data structures work well mainly in the development of application software.	7. Non-linear data structures work mainly well in image processing and Artificial Intelligence.

d) List any four applications of linked list.

Ans:

1. Implementation of stacks and queues
2. Performing arithmetic operations on long integers
3. Previous and next page in a web browser
4. Polynomial Manipulation representation
5. Symbol table creation
6. Mailing list
7. Memory management
8. Radix sort
9. Garbage collection

4		Attempt any three:	12 Marks
	a)	**Find the position of element 65 using binary search method in an Array 'A'.** **Given below:Show each step.** **A=(23,12,5,29,10,65,55,70} Write complete procedure.**	

Ans:

Given List, A = {23,12,5,29,10,65,55,70}

Given List is unsorted, so first sort the given List in ascending order.

A = {5,10,12,23,29,55,65,70}

Index	0	1	2	3	4	5	6	7
Data	5	10	12	23	29	55	65	70

search element - 65

step1 : set first = 0, last = 7, mid = $\frac{0+7}{2} = 3$

search element 65 compare with 23

Index	0	1	2	3	4	5	6	7
Data	5	10	12	23	29	55	65	70

No match, 65 is greater than 23, so search in right sublist only.

i.e

3	4	5	6	7
23	29	55	65	70

step2 : first = 3 last = 7 mid = $\frac{3+7}{2} = \frac{10}{2} = 5$

search element 65 is compare with 55

Index	3	4	5	6	7
Data	23	29	55	65	70

No match, 65 is greater than 55, so search in right sublist only

i.e

5	6	7
55	65	70

step 3 : first = 5
last = 7
mid = $\frac{5+7}{2} = 6$

search element 65 is compare with mid element 65

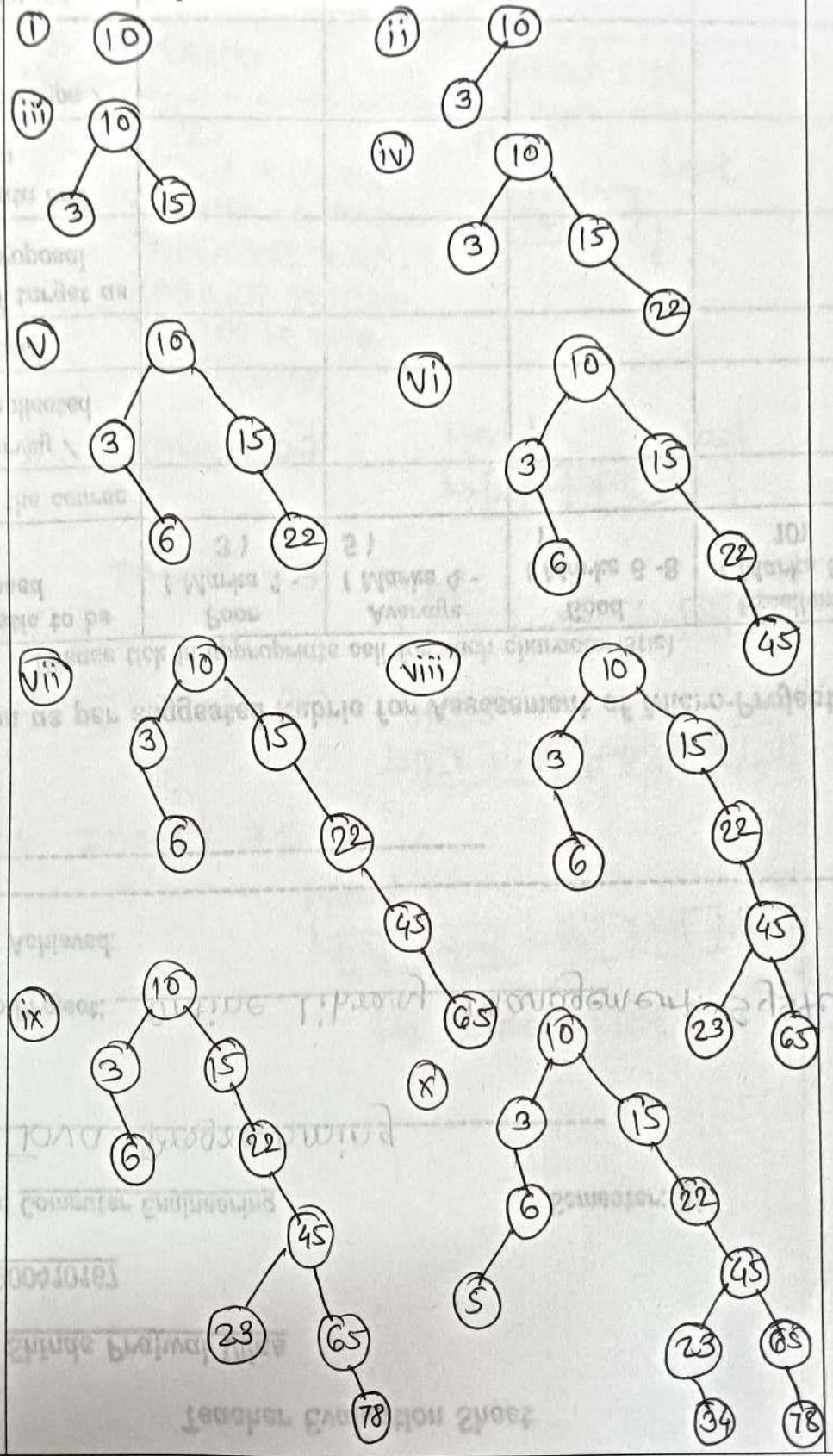
Match found at index 6, return

Index	0	1	2	3	4	5	6	7
Data	5	10	12	23	29	55	65	70

↑ match found

b) **Construct a binary search tree for following elements:**
10,3,15,22,6,45,65,23,78,34,5
show each step of construction of BSP.

Ans:



c) **Construct a singly linked list using data field:22,27,86,48, 75 and show procedure step-by-step with the help of diagram start to end.**

Ans:

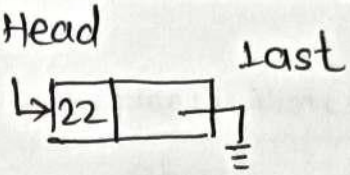
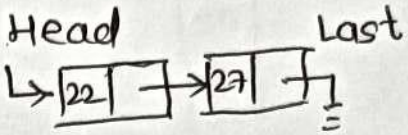
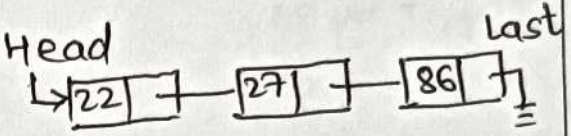
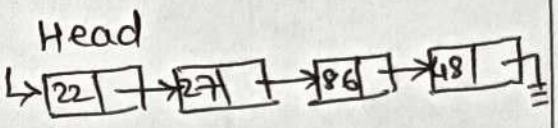
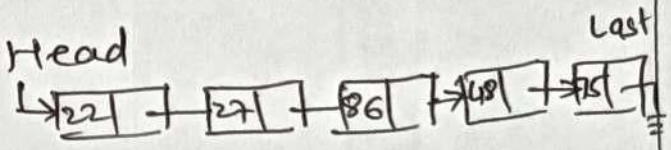
SR No.	Data	Linked List
1.	22 'Head pointer points to the first node and two last pointer points to the last node	
2.	Insert 27	
3.	Insert 86	
4.	Insert 48	
5	Insert 75	

fig. Linked List

d) **Show the effiect of PUSH and POP operation on to the stack of size 10.The stack contains 10,20,30,40,50 and 60 with 60 being at top of the stack.**

Show diagrammatically the effect of-

i) PUSH 55

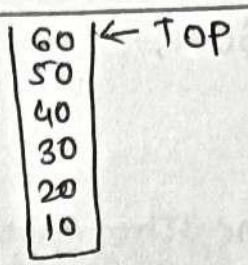
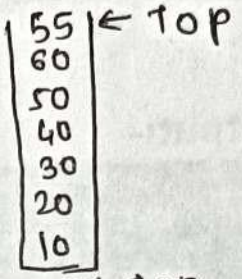
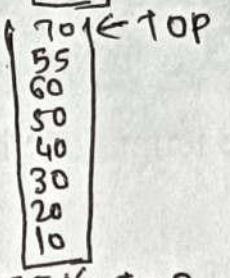
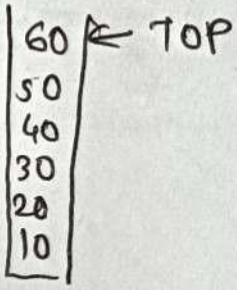
ii)PUSH 70

iii) POP

iv) POP

Sketch the final structure of stack after performing the above said Operations.

Ans:

St.No.	Action	stack
1.	Initial	
2.	PUSH 55	
3.	PUSH 70	
4.	POP	
5.	POP	

e) **Draw structure to following:**

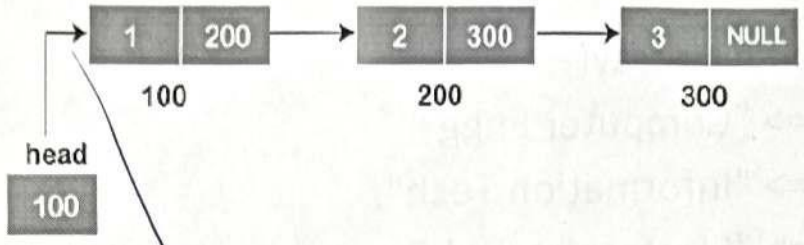
i) Singly linked list

ii) Doubly linked list

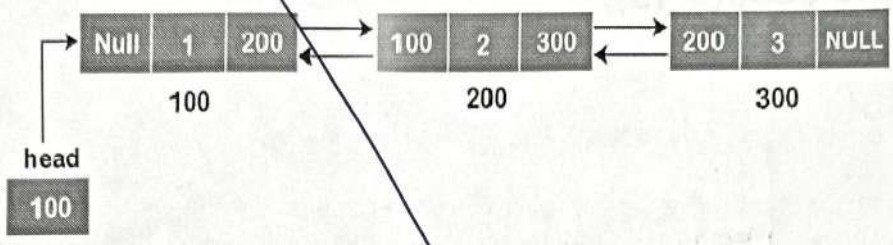
iii) Circular singly linked list

iv) Circular doubly linked list.

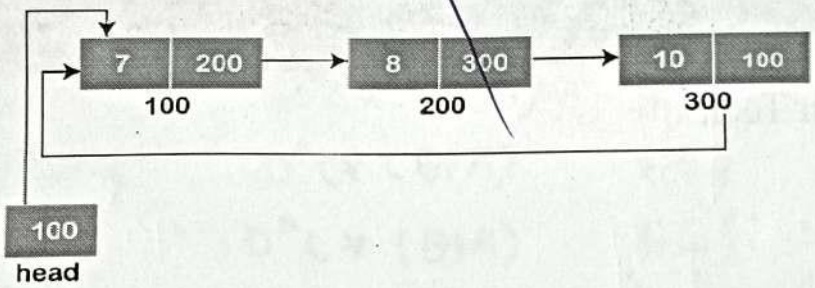
Ans: **i) Singly linked list:**



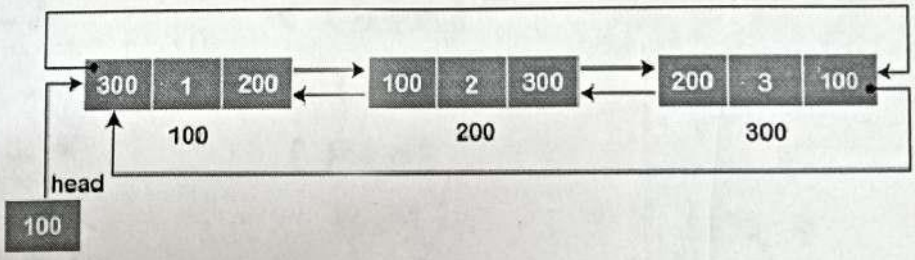
ii) Doubly linked list:



iii) Circular singly linked list:



iv) Circular doubly linked list.:



5		Attempt Any Two :	12 Marks
	a)	Convert the following infix expression into prefix using stack in tabular form : (A+B)*C^D-(E/F)	
	Ans:		

Infix expression - (A+B)*C^D-(E/F)

Infix to prefix :

step1 : Reversing the expression, we get
(F/E) - D^C * (B+A)

step2 : converting to postfix :

Sr.No.	stack	Expression	output
1.	Empty	(F/E)-D^C*(B+A)	-
2.	(	F/E)-D^C*(B+A)	-
3	(	/E)-D^C*(B+A)	F
4	(/	E)-D^C*(B+A)	F
5	(/	)-D^C*(B+A)	FE
6	(/	-D^C*(B+A)	FE/
7	Empty	-D^C*(B+A)	FE/
8	-	D^C*(B+A)	FE/
9.	-	^C*(B+A)	FE/D
10	-^	C*(B+A)	FE/D
11	-^	*(B+A)	FE/DC
12	-	*(B+A)	FE/DC↑
13	-*	(B+A)	FE/DC↑
14	-*(	B+A)	FE/DC↑
15	-*(	+A)	FE/DC↑B
16	-*(+	A)	FE/DC↑B
17	-*(+	)	FE/DC↑BA
18.	-*(+)	Empty	FE/DC↑BA+
19	-*	Empty	FE/DC↑BA+*-

step3 :- Reversing the expression, we get prefix expression : -*+AB↑CD/EF

b) **For a given Binary tree write in-order, pre-order and post-order traversal.**

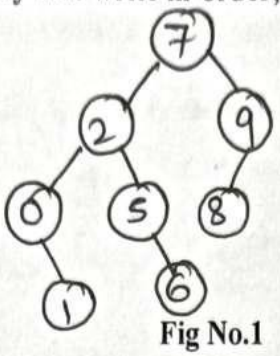


Fig No.1

Ans:

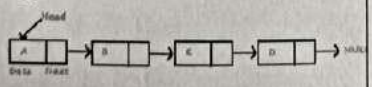
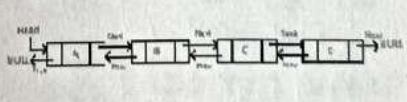
In order Traversal :
0, 1, 2, 5, 6, 7, 8, 9

Pre-order Traversal :
7, 2, 0, 1, 5, 6, 9, 8

Post order Traversal :
1, 0, 6, 5, 2, 8, 9, 7

c) **Distinguish between singly linked list and doubly linked list (Minimum Six Points each)**

Ans:

Singly linked list	Doubly linked list
1. Singly linked list nodes contains 2 field -data field and next link field.	1. Doubly linked list nodes contains 3 fields -data field, a previous link field and a next link field.
2. Traversal is possible in one direction only.	2. Traversal is possible in one direction only.
3. A singly linked list consumes less memory as compared to the doubly linked list.	3. The doubly linked list consumes more memory as compared to the singly linked list.
4. In singly linked lists, the complexity of insertion and deletion is O(n).	4. In doubly linked lists, the complexity of insertion and deletion is O(1).
5. We mostly prefer a singly linked list for the execution of stacks.	5. We can use a doubly linked list to execute binary trees, heaps and stacks.
6. 	6. 

6.		Attempt Any Two :	12 Marks

a) **Sort the following numbers in ascending order using selection sort.**
{35,14,5,102,61,10} and write the output after each iteration.

Ans:

Pass	Input/ output	Element	operation
Pass 1	Input	35 | 14 | 5 | 102 | 61 | 10	min = 5 swap 35 with 5
	output	5 | 14 | 35 | 102 | 61 | 10	After pass 1 smallest element 5 is at its proper position
Pass 2	Input	5 | 14 | 35 | 102 | 61 | 10	min = 10 swap 14 with 10
	output	5 | 10 | 35 | 102 | 61 | 14	After pass 2 smallest element 10 at its proper position
Pass 3	Input	5 | 10 | 35 | 102 | 61 | 14	min = 14 swap 35 with 14
	output	5 | 10 | 14 | 102 | 61 | 35	After pass 3 smallest element 14 is at its proper position.
Pass 4	Input	5 | 10 | 14 | 102 | 61 | 35	min = 35 swap 102 with 35
	output	5 | 10 | 14 | 35 | 61 | 102	After pass 4 smallest element 35 is at its proper position
Pass 5 :	Input	5 | 10 | 14 | 35 | 61 | 102	min = 61 No swap
	output	5 | 10 | 14 | 35 | 61 | 102	After pass 5 61 is its proper position So NO swap.

b) **Define the term recursion. Write a program in C to perform multiplication of two numbers using recursion.**

Ans: **Definition:** Recursion is the process of calling function by itself. A recursive function body contains function call statement that calls itself repeatedly.

Program:

```
#include<stdio.h>
#include<conio.h>
int multiply(int m,int n)
{
 if(m==0||n==0)
 return 0;
 return(m+multiply(m,n-1));
 }
 void main()
 {
  int m,n,result;
  printf("\n enter the two number");
  scanf("%d%d",&m,&n);
  result=multiply(m,n);
  printf("\n%dx%d=%d",m,n,result);
  getch();
  }
```

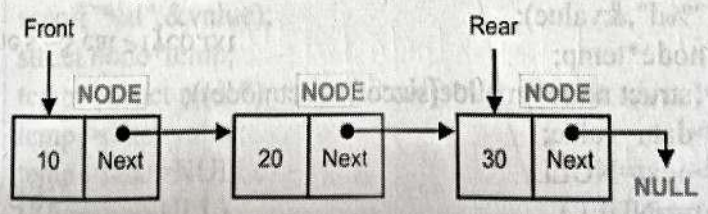
c) **Describe queue implementation using linked list.**

Ans: In linked Iist implementation of a queue, the last inserted node is always pointed by rear and the first node is always pointed by front.

Queue Representation with Linked List :

```
struct Node
{
int data;
struct Node *next;
}*front,*rear;
```

The Enqueue function will add the element at the end of the linked list. The Dequeue function will remove the first element from the queue. The traverse operation checks. whether the list is empty before trraversing the list. It will traverse all the elements, while temp-> next is not equal to NULL. Using loop, then it will print last elements data where temp->next=NULL.



```
#include<stdio.h>
#include<conio.h>
#include<stdlib.h>
#include<process.h>
struct node
{
int data;
struct node*next;
}*front=NULL,*rear=NULL;
void enqueue();
void dequeue();
void display();
int main()
{
int choice;
printf("\n**queue using linked list**\n");
printf("1.insert\n 2. delete\n 3.display\n 4.exit\n");
while(1)
{
printf("\n enter your choice:(1.insert 2.delete 3.display 4.exit)");
scanf("%d",&choice);
switch(choice)
{
case 1:enqueue();
break;
case 2:dequeue();
break;
case 3:display();
break;
case 4:exit(0);
default:
printf("wrong choice");
}
}
}
void enqueue()
{
int value;
printf("enter the element to be inserted:");
scanf("%d",&value);
struct node*temp;
temp=(struct node*)malloc(sizeof(struct node));
temp->data=value;
temp->next=NULL;
if(front==NULL)
```

```
front=rear=temp;
else
{
rear->next=temp;
rear=temp;
}
}
void dequeue()
{
if(front==NULL)
printf("\n queue is empty!!\n");
else
{
struct node*temp=front;
front=front->next;
printf("\n deleted element :%d\n",temp->data);
free(temp);
}
}
void display()
{
if(front==NULL)
printf("\n queue is empty!!");
else
{
struct node*temp=front;
while(temp->next!=NULL)
{
printf("%d",temp->data);
temp=temp->next;
}
printf("%d",temp->data);
}
}
```